

Ruby on Rails

Ruby on Rails (simplified as **Rails**) is a server-side web application framework written in Ruby under the MIT License. Rails is a model–view–controller (MVC) framework, providing default structures for a database, a web service, and web pages. It encourages and facilitates the use of web standards such as JSON or XML for data transfer and HTML, CSS and JavaScript for user interfacing. In addition to MVC, Rails emphasizes the use of other well-known software engineering patterns and paradigms, including convention over configuration (CoC), don't repeat yourself (DRY), and the active record pattern.^[4]

Ruby on Rails' emergence in 2005 greatly influenced web app development, through innovative features such as seamless database table creations, migrations, and scaffolding of views to enable rapid application development. Ruby on Rails' influence on other web frameworks remains apparent today, with many frameworks in other languages borrowing its ideas, including Django in Python; Catalyst in Perl; Laravel, CakePHP and Yii in PHP; Grails in Groovy; Phoenix in Elixir; Play in Scala; and Sails.js in Node.js.

Well-known sites that use Ruby on Rails include Airbnb, Archive of Our Own, Crunchbase, Dribbble,^[5] GitHub,^[6] Twitch^[7] and Shopify.

History

David Heinemeier Hansson extracted Ruby on Rails from his work on the project management tool Basecamp at the web application company 37signals.^[8] Hansson first released Rails as open source in July 2004, but did not share commit rights to the project until February 2005. In August 2006, the framework reached a milestone when Apple announced that it would ship Ruby on Rails with Mac OS X v10.5 "Leopard",^[9] which was released in October 2007.

Rails version 2.3 was released on 15 March 2009, with major new developments in templates, engines, Rack and nested model forms. Templates enable the developer to generate a skeleton application with custom gems and configurations. Engines give developers the ability to reuse application pieces complete with routes, view paths and models. The Rack web server interface and Metal allow one to write optimized pieces of code that route around Action Controller.^[10]

Ruby on Rails



Original author	<u>David Heinemeier Hansson</u>
Initial release	July 2004 ^[1]
Stable release	8.0: 8.1.3 ^[2] / 24 March 2026 ^{}
Written in	<u>Ruby</u>
Size	20.2 MB ^[3]
Type	<u>Web application framework</u>
License	<u>MIT License</u>
Website	<u>rubyonrails.org</u> (<u>https://rubyonrails.org</u>)
Repository	<u>github.com/rails/rails</u> (<u>https://github.com/rails/rails</u>)

On 23 December 2008, Merb, another web application framework, was launched, and Ruby on Rails announced it would work with the Merb project to bring "the best ideas of Merb" into Rails 3, ending the "unnecessary duplication" across both communities.^[11] Merb was merged with Rails as part of the Rails 3.0 release.^{[12][13]}

Rails 3.1 was released on 31 August 2011, featuring Reversible Database Migrations, Asset Pipeline, Streaming, jQuery as default JavaScript library and newly introduced CoffeeScript and Sass into the stack.^[14]

Rails 3.2 was released on 20 January 2012 with a faster development mode and routing engine (also known as Journey engine), Automatic Query Explain and Tagged Logging.^[15] Rails 3.2.x is the last version that supports Ruby 1.8.7.^[16] Rails 3.2.12 supports Ruby 2.0.^[17]

Rails 4.0 was released on 25 June 2013, introducing Russian Doll Caching, Turbolinks, and Live Streaming as well as making Active Resource, Active Record Observer and other components optional by splitting them as gems.^[18]

Rails 4.1 was released on 8 April 2014, introducing Spring, Variants, Enums, Mailer previews, and secrets.yml.^[19]

Rails 4.2 was released on 19 December 2014, introducing Active Job, asynchronous emails, Adequate Record, Web Console, and foreign keys.^[20]

Rails 5.0 was released on 30 June 2016, introducing Action Cable, API mode, and Turbolinks 5.^[21]

Rails 5.0.0.1 was released on 10 August 2016, with Exclusive use of rails CLI over Rake and support for Ruby version 2.2.2 and above.

Rails 5.1 was released on 27 April 2017, introducing JavaScript integration changes (management of JavaScript dependencies from NPM via Yarn, optional compilation of JavaScript using Webpack, and a rewrite of Rails UJS to use vanilla JavaScript instead of depending on jQuery), system tests using Capybara, encrypted secrets, parameterized mailers, direct & resolved routes, and a unified form_with helper replacing the form_tag/form_for helpers.^[22]

Rails 5.2 was released on 9 April 2018, introducing new features that include ActiveStorage, built-in Redis Cache Store, updated Rails Credentials and a new DSL that allows for configuring a Content Security Policy for an application.^[23]

Rails 5.2.2 was released on 4 December 2018, introducing numerous bug fixes and several logic improvements.^[24]

Rails 6.0 was released on 16 August 2019, making Webpack default, adding mailbox routing, a default online rich-text editor, parallel testing, multiple database support, mailer routing and a new autoloader.^[25]

Rails 6.1 was released on 9 December 2020, adding per-database connection switching, horizontal database sharding, eager loading of all associations, Delegated Types as an alternative to single-table inheritance, asynchronous deletion of associations, error objects, and other improvements and bug fixes.^[26]

Rails 7.0 was released on 15 December 2021, replacing Node.js and Webpack with import maps for JavaScript management by default, replacing Turbolinks with a combination of Turbo and Stimulus, adding at-work encryption into Active Record, using Zeitwerk exclusively for code loading, and more.^[27]

Rails 7.1 was released on 5 October 2023, Dockerfiles support using Kamal in order to deploy your application, authentication improvements, and now including support for bun.^[28]

Ruby on Rails 8.0.0 was released on 8 November 2024. This major release introduces fundamental shifts in Rails development, enabling individual developers to host and manage their applications independently without the need for a Platform-as-a-service. The update focuses on empowering single developers to handle all aspects of application deployment and management seamlessly.^[29]

Ruby on Rails 8.1.0 was released on October 24, 2025. Major features include Active Job Continuations, Structured Event Reporting, Local CI, and Markdown rendering.^[30]

Version History

Version	Release date	Compatible Ruby version(s) ^{[31][32]}
1.0 ^[33]	13 December 2005	1.8.6
1.2 ^[34]	19 January 2007	1.8.6
2.0 ^[35]	7 December 2007	1.8.6
2.1 ^[36]	31 May 2008	1.8.6
2.2 ^[37]	21 November 2008	1.8.7 recommended; 1.8.6 possible
2.3 ^[38]	16 March 2009	1.8.7 recommended; 1.8.6 and 1.9.1 possible
3.0 ^[39]	29 August 2010	1.9.3 recommended; 1.8.7 and 1.9.2 possible
3.1 ^[40]	31 August 2011	1.9.3 recommended; 1.8.7 and 1.9.2 possible
3.2 ^[41]	20 January 2012	1.9.3 recommended; 1.8.7 and 1.9.2 possible
4.0 ^[42]	25 June 2013	2.0 preferred; 1.9.3 or newer required
4.1 ^[19]	8 April 2014	2.0 preferred; 1.9.3 or newer required
4.2 ^[20]	19 December 2014	2.0 preferred; 1.9.3 or newer required
5.0 ^[21]	30 June 2016	2.2.2 or newer
5.1 ^[22]	10 May 2017	2.2.2 or newer
5.2 ^[23]	9 April 2018	2.2.2 or newer
6.0 ^[25]	16 August 2019	2.5.0 or newer
6.1 ^[26]	9 December 2020	2.5.0 or newer
7.0 ^[27]	15 December 2021	2.7.0 or newer
7.1 ^[43]	5 October 2023	2.7.0 or newer
7.2 ^[44]	10 August 2024	3.1.0 or newer
8.0 ^[45]	7 November 2024	3.2.0 or newer
8.1^[46]	24 October 2025	3.2.0 or newer
Unsupported Supported Latest version Future version		

A revised maintenance policy^[47] was instituted in October 2024.^[48]

Technical overview

Ruby on Rails evolves radically from release to release exploring the use of new technologies and adopting new standards on the Internet. Some features are very stable in Ruby on Rails while some are replaced in favour of new techniques.

Model-view-controller pattern

The model-view-controller (MVC) pattern is the fundamental structure to organize application programming.

In a default configuration, a *model* in the Ruby on Rails framework maps to a table in a database and to a Ruby file. For example, a model class *User* will usually be defined in the file 'user.rb' in the app/models directory, and linked to the table 'users' in the database. While developers are free to ignore this convention and choose differing names for their models, files, and database table, this is not common practice and is usually discouraged in accordance with the "convention-over-configuration" philosophy.^[49]

A *controller* is a server-side component of Rails that responds to external requests from the web server to the application, by determining which view file to render. The controller may also have to query one or more models for information and pass these on to the view. For example, in an airline reservation system, a controller implementing a flight-search function would need to query a model representing individual flights to find flights matching the search, and might also need to query models representing airports and airlines to find related secondary data. The controller might then pass some subset of the flight data to the corresponding view, which would contain a mixture of static HTML and logic that use the flight data to create an HTML document containing a table with one row per flight. A controller may provide one or more actions. In Ruby on Rails, an action is typically a basic unit that describes how to respond to a specific external web-browser request. Also, note that the controller/action will be accessible for external web requests only if a corresponding route is mapped to it. Rails encourages developers to use RESTful routes, which include actions such as create, new, edit, update, destroy, show, and index. These mappings of incoming requests/routes to controller actions can be easily set up in the routes.rb configuration file.

A *view* in the default configuration of Rails is an erb file, which is evaluated and converted to HTML at run-time. Alternatively, many other templating systems can be used for views.

Ruby on Rails includes tools that make common development tasks easier "out-of-the-box", such as scaffolding that can automatically construct some of the models and views needed for a basic website.^[50] Also included are WEBrick, a simple Ruby web server that is distributed with Ruby, and Rake, a build system, distributed as a gem. Together with Ruby on Rails, these tools provide a basic development environment.

HTTP servers

Ruby on Rails is most commonly not connected to the Internet directly, but through some front-end web server. Mongrel was generally preferred over WEBrick in the early days, but it can also run on Lighttpd, Apache, Cherokee, Hiawatha, Nginx (either as a module – Phusion Passenger for example – or via CGI, FastCGI or mod ruby), and many others. From 2008 onward, Passenger replaced Mongrel as the most-used web server for Ruby on Rails.^[51] Ruby is also supported natively on IBM i.^[52]

JavaScript

Ruby on Rails is also noteworthy for its extensive use of the JavaScript libraries Prototype and Script.aculo.us for scripting Ajax actions.^[53] Ruby on Rails 3.0 separates the markup of the page (which defines the structure of the page) from scripting (which determines functionality or logic of the page). As

of version 7.0, new Ruby on Rails applications come with the Hotwire (<https://hotwired.dev/>) family of JavaScript libraries installed by default.

Web services

Since version 2.0, Ruby on Rails offers both HTML and XML as standard output formats. The latter is the facility for RESTful web services.

CSS

Rails 3.1 introduced Sass as standard CSS templating.

Template

By default, the server uses Embedded Ruby in the HTML views, with files having an `html.erb` extension. Rails supports swapping-in alternative templating languages, such as HAML and Mustache.

Ruby versions

Ruby on Rails 3.0 has been designed to work with Ruby 1.8.7, Ruby 1.9.2, and JRuby 1.5.2+; earlier versions are not supported.^[54]

Ruby on Rails 3.2 is the last series of releases that support Ruby 1.8.7.

Framework structure

Ruby on Rails is separated into various packages, namely ActiveRecord (an object-relational mapping system for database access), Action Pack, Active Support and Action Mailer. Prior to version 2.0, Ruby on Rails also included the Action Web Service package that is now replaced by Active Resource. Apart from standard packages, developers can make plugins to extend existing packages. Earlier Rails supported plugins within their own custom framework; version 3.2 deprecates these in favor of standard Ruby "gems".^[55]

Deployment

Ruby on Rails is often installed using RubyGems, a package manager^[56] which is included with current versions of Ruby. Many free Unix-like systems also support installation of Ruby on Rails and its dependencies through their native package management system.

Ruby on Rails is typically deployed with a database server such as MySQL or PostgreSQL, and a web server such as Apache running the Phusion Passenger module.

Philosophy and design

Ruby on Rails is intended to emphasize Convention over Configuration (CoC), and the Don't Repeat Yourself (DRY) principle.

The Rails Doctrine (rubyonrails.org/doctrine (<https://rubyonrails.org/doctrine>)) is an enduring enabler that guides the philosophy, design, and implementation of the Ruby on Rails framework.

"Convention over Configuration" means a developer only needs to specify unconventional aspects of the application. For example, if there is a class *Sale* in the model, the corresponding table in the database is called *sales* by default. It is only if one deviates from this convention, such as calling the table "products sold", that the developer needs to write code regarding these names. Generally, Ruby on Rails conventions lead to less code and less repetition.^[57]

"Don't repeat yourself" means that information is located in a single, unambiguous place. For example, using the ActiveRecord module of Rails, the developer does not need to specify database column names in class definitions. Instead, Ruby on Rails can retrieve this information from the database based on the class name.

"Fat models, skinny controllers" means that most of the application logic should be placed within the model while leaving the controller as light as possible.

HTML Over The Wire (Hotwire),^{[58][59]} Conceptual compression,^{[60][61]} and robust security^[62] mark Rails 7.0's approach to the One person framework.^[a]

Trademarks

In March 2007, David Heinemeier Hansson applied to register three Ruby on Rails-related trademarks with the USPTO. These applications concern the phrase "RUBY ON RAILS",^[64] the word "RAILS",^[65] and the official Rails logo.^[66] In the summer of 2007, Hansson denied the publisher Apress permission to use the Ruby on Rails logo on the cover of a new Ruby on Rails book written by some authoritative community members. The episode gave rise to a polite protest in the Ruby on Rails community.^{[67][68]} In response to this criticism, Hansson replied:

I only grant promotional use [of the Rails logo] for products I'm directly involved with. Such as books that I've been part of the development process for or conferences where I have a say in the execution. I would most definitely seek to enforce all the trademarks of Rails.^[67]

The trademark of the logo was cancelled on 25 October 2019.^[66]

Reception

Scalability

In earlier days, Rails running on Matz's Ruby Interpreter (the *de facto* reference interpreter for Ruby) had been criticized for issues with scalability.^[69] These critiques often mentioned various Twitter outages in 2007 and 2008, which spurred Twitter's partial transition to Scala (which runs on the Java Virtual Machine) for their queueing system and other middleware.^{[70][71]} The user interface aspects of the site continued to run Ruby on Rails^[72] until 2011 when it was replaced due to concerns over performance.^[73]

On the other hand, many Rails business application developers relied on system architecture design, including choices of database engine, cache configuration, and servers, to tackle scalability issues. The original author of Rails, David Heinemeier Hansson, criticized Twitter, saying that their problems scaling were the consequences of their own poor architectural decisions and not the fault of Rails. According to Hansson, blaming Rails for their troubles while making no contributions to the framework is ungrateful and unjust.^[74]

In 2011, [Gartner Research](#) noted that despite criticisms and comparisons to Java, many high-profile consumer web firms are using Ruby on Rails to build scalable web applications. Some of the largest sites running Ruby on Rails include [Airbnb](#), [Cookpad](#), [GitHub](#), [GitLab](#),^[75] [Scribd](#), [Shopify](#), and [Basecamp](#).^[76] As of January 2016, it is estimated that more than 1.2 million web sites are running Ruby on Rails.^{[77][78]}

Security

In March 2012, security researcher Egor Homakov discovered a [mass assignment vulnerability](#) that allowed certain Rails applications to be remotely exploited, and demonstrated it by non-maliciously hacking [GitHub](#) after his earlier attempts at responsible disclosure were dismissed.^[79]

On 24 September 2013, a session cookie persistence security flaw was reported in Ruby on Rails. In a default configuration, the entire session hash is stored within a session cookie known as *CookieStore*, allowing any authenticated session possessing the session cookie to log in as the target user at any time in the future. As a workaround, administrators are advised to configure cookies to be stored on the server using mechanisms such as *ActiveRecordStore*.^[80]

Researchers Daniel Jackson and Joseph Near developed a data debugger they called "Space" that can analyze the data access of a Rails program and determine if the program properly adheres to rules regarding access restrictions. On 15 April 2016, Near reported that an analysis of 50 popular Web applications using Space uncovered 23 previously unknown security flaws.^[81]

See also

- [Comparison of server-side web frameworks](#)
- [Library \(computing\)](#)
- [List of rich web application frameworks](#)

Notes

- a. Ruby on Rails version 7.0 allows the One-Person Framework (of which the Single page application is one approach) ^[63]

References

1. "[ANN] Rails 0.5.0: The end of vaporware!" (<https://rubytalk.org/t/ann-rails-0-5-0-the-end-of-vaporware/12744>). *rubytalk.org*. 24 July 2004. Archived (<https://web.archive.org/web/20240801161553/https://rubytalk.org/t/ann-rails-0-5-0-the-end-of-vaporware/12744>) from the original on 1 August 2024. Retrieved 1 August 2024.

2. "Release (8.1.3)" (<https://github.com/rails/rails/releases/tag/v8.1.3>). Retrieved 26 March 2026.
3. "Release 8.1.3 · rails/rails · GitHub" (<https://github.com/rails/rails/releases/tag/v8.1.3>). *GitHub*. Retrieved 26 March 2026.
4. "Getting Started with Rails: What Is Rails?" (http://guides.rubyonrails.org/getting_started.html#what-is-rails-questionmark). *guides.rubyonrails.org*. Archived (https://web.archive.org/web/20160429222517/http://guides.rubyonrails.org/getting_started.html#what-is-rails-questionmark) from the original on 29 April 2016. Retrieved 10 August 2014.
5. Goswami, Shubham. "Django vs Ruby on Rails – The Choice for Mobile App Development" (<https://web.archive.org/web/20210131010549/https://appsearch.org/blog/django-vs-ruby-on-rails/>). *appsearch.org*. Archived from the original (<https://appsearch.org/blog/django-vs-ruby-on-rails/>) on 31 January 2021. Retrieved 28 November 2020.
6. Conrad, Adam (3 August 2021). "Episode 471: Jason Meller on Choosing the Right Tech Stack for a Greenfield Project" (<https://www.se-radio.net/2021/08/episode-471-jason-meller-on-choosing-the-right-tech-stack-for-a-greenfield-project/>). *SE-Radio.net*. IEEE. Archived (<https://web.archive.org/web/20211109055008/https://www.se-radio.net/2021/08/episode-471-jason-meller-on-choosing-the-right-tech-stack-for-a-greenfield-project/>) from the original on 9 November 2021. Retrieved 9 November 2021.
7. "Breaking the Monolith at Twitch: Part One" (<https://blog.twitch.tv/en/2022/03/30/breaking-the-monolith-at-twitch/#:~:text=Twitch.tv%20launched%20on%20June,initially%20worked%20well%20for%20Twitch.>).
8. Grimmer, Lenz-- (February 2006). "Interview with David Heinemeier Hansson from Ruby on Rails" (<https://web.archive.org/web/20130225091835/http://dev.mysql.com/tech-resources/interviews/david-heinemeier-hansson-rails.html>). MySQL AB. Archived from the original (<http://dev.mysql.com/tech-resources/interviews/david-heinemeier-hansson-rails.html>) on 25 February 2013. Retrieved 8 June 2008.
9. Hansson, David (7 August 2006). "Ruby on Rails will ship with OS X 10.5 (Leopard)" (<http://weblog.rubyonrails.org/2006/8/7/ruby-on-rails-will-ship-with-os-x-10-5-leopard>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20060813093250/http://weblog.rubyonrails.org/2006/8/7/ruby-on-rails-will-ship-with-os-x-10-5-leopard>) from the original on 13 August 2006. Retrieved 8 June 2008.
10. Hansson, David (16 March 2009). "Rails 2.3: Templates, Engines, Rack, Metal, much more!" (<http://weblog.rubyonrails.org/2009/3/16/rails-2-3-templates-engines-rack-metal-much-more>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20090321021909/http://weblog.rubyonrails.org/2009/3/16/rails-2-3-templates-engines-rack-metal-much-more>) from the original on 21 March 2009. Retrieved 22 March 2009.
11. "The day Merb joined Rails" (<https://web.archive.org/web/20130209055716/http://rubyonrails.org/merb>). *rubyonrails.org*. 27 December 2008. Archived from the original (<http://rubyonrails.org/merb>) on 9 February 2013.
12. "Ruby on Rails 3.0 Release Notes — Ruby on Rails Guides" (http://edgeguides.rubyonrails.org/3_0_release_notes.html). *edgeguides.rubyonrails.org*. Retrieved 24 May 2017.
13. "Ruby on Rails 3.0 goes modular" (<http://www.sdtimes.com/link/34122>). *sdtimes.com*. 10 February 2010. Archived (<https://web.archive.org/web/20100924023725/http://www.sdtimes.com/link/34122>) from the original on 24 September 2010. Retrieved 6 August 2010.
14. "Ruby on Rails 3.1 Release Notes" (http://guides.rubyonrails.org/3_1_release_notes.html). *guides.rubyonrails.org*. 1 September 2012. Archived (https://web.archive.org/web/20120827182929/http://guides.rubyonrails.org/3_1_release_notes.html) from the original on 27 August 2012. Retrieved 3 September 2012.
15. "Ruby on Rails 3.2 Release Notes" (http://guides.rubyonrails.org/3_2_release_notes.html). *guides.rubyonrails.org*. 1 September 2012.

16. David (1 September 2012). "Rails/master is now 4.0.0.beta" (<http://weblog.rubyonrails.org/2011/12/20/rails-master-is-now-4-0-0-beta/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20120903112624/http://weblog.rubyonrails.org/2011/12/20/rails-master-is-now-4-0-0-beta/>) from the original on 3 September 2012. Retrieved 3 September 2012.
17. Rails 3.2.x is now compatible with Ruby 2.0.0 by sikachu · Pull Request #9406 · rails/rails · GitHub (<https://github.com/rails/rails/pull/9406>) Archived (<https://web.archive.org/web/20170729100259/https://github.com/rails/rails/pull/9406>) 29 July 2017 at the Wayback Machine. Github.com. Retrieved on 2014-05-30.
18. Dhh (25 June 2013). "Rails 4.0: Final version released!" (<http://weblog.rubyonrails.org/2013/6/25/Rails-4-0-final/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20160427003834/http://weblog.rubyonrails.org/2013/6/25/Rails-4-0-final/>) from the original on 27 April 2016. Retrieved 24 May 2017.
19. Dhh (8 April 2014). "Rails 4.1.0: Spring, Variants, Enums, Mailer previews, secrets.yml" (<http://weblog.rubyonrails.org/2014/4/8/Rails-4-1/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20151208160905/http://weblog.rubyonrails.org/2014/4/8/Rails-4-1/>) from the original on 8 December 2015. Retrieved 8 April 2014.
20. Dhh (19 December 2014). "Rails 4.2: Active Job, Asynchronous Mails, Adequate Record, Web Console, Foreign Keys" (<http://weblog.rubyonrails.org/2014/12/19/Rails-4-2-final/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20151209162953/http://weblog.rubyonrails.org/2014/12/19/Rails-4-2-final/>) from the original on 9 December 2015. Retrieved 5 December 2015.
21. Dhh (30 June 2016). "Rails 5.0: Action Cable, API mode, and so much more" (<http://weblog.rubyonrails.org/2016/6/30/Rails-5-0-final/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20160630235010/http://weblog.rubyonrails.org/2016/6/30/Rails-5-0-final/>) from the original on 30 June 2016. Retrieved 19 November 2016.
22. Dhh (27 April 2017). "Rails 5.1: Loving JavaScript, System Tests, Encrypted Secrets, and more" (<http://weblog.rubyonrails.org/2017/4/27/Rails-5-1-final/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20170427222012/http://weblog.rubyonrails.org/2017/4/27/Rails-5-1-final/>) from the original on 27 April 2017. Retrieved 10 May 2017.
23. "Rails 5.2.0 FINAL: Active Storage, Redis Cache Store, HTTP/2 Early Hints, CSP, Credentials" (<https://weblog.rubyonrails.org/2018/4/9/Rails-5-2-0-final/>). *weblog.rubyonrails.org*. 9 April 2018. Archived (<https://web.archive.org/web/20200726094725/https://weblog.rubyonrails.org/2018/4/9/Rails-5-2-0-final/>) from the original on 26 July 2020. Retrieved 9 April 2018.
24. rafaelfranca (4 December 2018). "Rails 5.2.2 has been released!" (<https://weblog.rubyonrails.org/2018/12/4/Rails-5-2-2-has-been-released/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20181205141007/https://weblog.rubyonrails.org/2018/12/4/Rails-5-2-2-has-been-released/>) from the original on 5 December 2018. Retrieved 4 January 2019.
25. dhh (19 August 2019). "Rails 6.0: Action Mailbox, Action Text, Multiple DBs, Parallel Testing, Webpacker by default, and Zeitwerk" (<https://weblog.rubyonrails.org/2019/8/15/Rails-6-0-final-release/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20191202085218/https://weblog.rubyonrails.org/2019/8/15/Rails-6-0-final-release/>) from the original on 2 December 2019. Retrieved 12 November 2019.
26. eileencodes (9 December 2020). "Rails 6.1: Horizontal Sharding, Multi-DB Improvements, Strict Loading, Destroy Associations in Background, Error Objects, and more!" (<https://weblog.rubyonrails.org/2020/12/9/Rails-6-1-0-release/>). *weblog.rubyonrails.org*. Retrieved 28 December 2020.
27. dhh (15 December 2021). "Rails 7.0: Fulfilling a vision" (<https://rubyonrails.org/2021/12/15/Rails-7-fulfilling-a-vision/>). *weblog.rubyonrails.org*. Retrieved 16 December 2021.

28. "Rails 7.1: Dockerfiles, BYO Authentication, More Async Queries, and more!" (<https://rubyonrails.org/2023/10/5/Rails-7-1-0-has-been-released>). *Ruby on Rails*. Archived (<https://web.archive.org/web/20231005180516/https://rubyonrails.org/2023/10/5/Rails-7-1-0-has-been-released>) from the original on 5 October 2023. Retrieved 8 October 2023.
29. "Rails 8.0 Released: Empowering Developers to Self-Host" (https://www.youtube.com/watch?v=-cEn_83zRFw). *YouTube*. 26 September 2024. Archived (https://web.archive.org/web/20241108000000/https://www.youtube.com/watch?v=-cEn_83zRFw) from the original on 8 November 2024. Retrieved 8 November 2024.
30. "Rails 8.1: Job continuations, structured events, local CI" (https://guides.rubyonrails.org/8_1_release_notes.html). 24 October 2025. Retrieved 6 November 2025.
31. "Ruby/Rails Compatibility Matrix" (<https://web.archive.org/web/20220818051110/https://dev.alot.com/articles/2012/03/ror-compatibility>). *dev.alot.com*. Archived from the original (<https://dev.alot.com/articles/2012/03/ror-compatibility>) on 18 August 2022. Retrieved 5 March 2021.
32. "Upgrading Ruby on Rails" (https://edgeguides.rubyonrails.org/upgrading_ruby_on_rails.html#ruby-versions). *edgeguides.rubyonrails.org*. Retrieved 5 March 2021.
33. "Rails 1.0: Party like it's one oh oh!" (<http://weblog.rubyonrails.org/2005/12/13/rails-1-0-party-like-its-one-oh-oh/>) Archived (<https://web.archive.org/web/20151217174626/http://weblog.rubyonrails.org/2005/12/13/rails-1-0-party-like-its-one-oh-oh/>) 17 December 2015 at the Wayback Machine. *weblog.rubyonrails.org*. Retrieved 9 June 2010.
34. "Rails 1.2: REST admiration, HTTP lovefest, and UTF-8 celebrations" (<http://weblog.rubyonrails.org/2007/1/19/rails-1-2-rest-admiration-http-lovefest-and-utf-8-celebrations>) Archived (<https://web.archive.org/web/20121102040417/http://weblog.rubyonrails.org/2007/1/18/rails-1-2-rest-admiration-http-lovefest-and-utf-8-celebrations/>) 2012-11-02 at the Wayback Machine. *weblog.rubyonrails.org*. Retrieved 9 June 2010.
35. "Rails 2.0: It's done!" (<http://weblog.rubyonrails.org/2007/12/7/rails-2-0-it-s-done>) Archived (<https://web.archive.org/web/20100528225206/http://weblog.rubyonrails.org/2007/12/7/rails-2-0-it-s-done>) 28 May 2010 at the Wayback Machine. *weblog.rubyonrails.org*. Retrieved 9 June 2010.
36. Rails 2.1: Time zones, dirty, caching, gem dependencies, caching, etc (<http://weblog.rubyonrails.org/2008/5/31/rails-2-1-time-zones-dirty-caching-gem-dependencies-caching-etc>) Archived (<https://web.archive.org/web/20121102040223/http://weblog.rubyonrails.org/2008/5/31/rails-2-1-time-zones-dirty-caching-gem-dependencies-caching-etc/>) 2012-11-02 at the Wayback Machine. *weblog.rubyonrails.org*. Retrieved 9 June 2010.
37. "Rails 2.2: i18n, HTTP validators, thread safety, JRuby/1.9 compatibility, docs" (<http://weblog.rubyonrails.org/2008/11/21/rails-2-2-i18n-http-validators-thread-safety-jruby-1-9-compatibility-docs>) Archived (<https://web.archive.org/web/20081202082934/http://weblog.rubyonrails.org/2008/11/21/rails-2-2-i18n-http-validators-thread-safety-jruby-1-9-compatibility-docs>) 2 December 2008 at the Wayback Machine. *weblog.rubyonrails.org*. Retrieved 9 June 2010.
38. "Rails 2.3: Templates, Engines, Rack, Metal, much more!" (<http://weblog.rubyonrails.org/2009/3/16/rails-2-3-templates-engines-rack-metal-much-more>) Archived (<https://web.archive.org/web/20090321021909/http://weblog.rubyonrails.org/2009/3/16/rails-2-3-templates-engines-rack-metal-much-more>) 21 March 2009 at the Wayback Machine. *weblog.rubyonrails.org*. Retrieved 9 June 2010.
39. "Rails 3.0: It's ready!" (<http://weblog.rubyonrails.org/2010/8/29/rails-3-0-it-s-done>) Archived (<https://web.archive.org/web/20100831063441/http://weblog.rubyonrails.org/2010/8/29/rails-3-0-it-s-done>) 31 August 2010 at the Wayback Machine. *weblog.rubyonrails.org*. Retrieved 30 August 2010.
40. "bumping to 3.1.0" (<https://github.com/rails/rails/commit/505e84599aff6abf719484636b0515e1ce2e2220>). *Github.com*. Archived (<https://web.archive.org/web/20160619140551/https://github.com/rails/rails/commit/505e84599aff6abf719484636b0515e1ce2e2220>) from the original on 19 June 2016. Retrieved 31 August 2011.

41. "Preparing for 3.2.0 release" (<https://github.com/rails/rails/commit/f36dcaf488b4357a52f43e3912628428956d351f>). *Github.com*. Archived (<https://web.archive.org/web/20170729100334/https://github.com/rails/rails/commit/f36dcaf488b4357a52f43e3912628428956d351f>) from the original on 29 July 2017. Retrieved 22 January 2012.
42. "Rails 4 in 30'" (<https://web.archive.org/web/20121204083513/http://blog.wyeworks.com/2012/10/29/rails-4-in-30-minutes/>). *blog.wyeworks.com*. Archived from the original (<http://blog.wyeworks.com/2012/10/29/rails-4-in-30-minutes/>) on 4 December 2012. Retrieved 15 December 2012.
43. França, Rafael. "Rails 7.1: Dockerfiles, BYO Authentication, More Async Queries, and more!" (<https://rubyonrails.org/2023/10/5/Rails-7-1-0-has-been-released>). *rubyonrails.org*. Archived (<https://web.archive.org/web/20231005085736/https://rubyonrails.org/2023/10/5/Rails-7-1-0-has-been-released>) from the original on 5 October 2023. Retrieved 5 October 2023.
44. França, Rafael. "Development of Rails 7.2 starts now" (<https://github.com/rails/rails/commit/fb6c6007d02114f272c506cb837fff1e7534f413>). *GitHub*. Archived (<https://web.archive.org/web/20231125014449/https://github.com/rails/rails/commit/fb6c6007d02114f272c506cb837fff1e7534f413>) from the original on 25 November 2023. Retrieved 26 September 2023.
45. "Rails 8.0: No PaaS Required" (<https://rubyonrails.org/2024/11/7/rails-8-no-paas-required>).
46. "Rails 8.1: Job continuations, structured events, local CI" (<https://rubyonrails.org/2025/10/2/rails-8-1>).
47. "Maintenance policy" (<https://rubyonrails.org/maintenance>).
48. "New Rails maintenance policy and end of maintenance announcements" (<https://rubyonrails.org/2024/10/15/new-maintenance-policy-and-eol-announcements>).
49. "The Rails Doctrine" (<https://rubyonrails.org/doctrine>). *Ruby on Rails*. Archived (<https://web.archive.org/web/20210208223044/https://rubyonrails.org/doctrine/>) from the original on 8 February 2021. Retrieved 7 June 2024.
50. Sean Lynch (15 December 2007). "fairleads: Rails 2.0 and Scaffolding Step by Step" (<http://fairleads.blogspot.com/2007/12/rails-20-and-scaffolding-step-by-step.html>). *Fairleads.blogspot.com*. Archived (<https://web.archive.org/web/20090529115259/http://fairleads.blogspot.com/2007/12/rails-20-and-scaffolding-step-by-step.html>?) from the original on 29 May 2009. Retrieved 24 May 2017.
51. "Ruby on Rails: Deploy" (<https://web.archive.org/web/20090707080544/http://rubyonrails.org/g/deploy>). *rubyonrails.org*. Archived from the original (<http://rubyonrails.org/deploy>) on 7 July 2009. Retrieved 3 December 2012.
52. Maxcer, Chris. "PowerRuby Brings Enterprise Ruby on Rails Support to IBM i" (<https://web.archive.org/web/20131009010101/http://iprodeveloper.com/application-development/powerruby-brings-enterprise-ruby-rails-support-ibm-i>). *iPro Developer*. Archived from the original (<http://iprodeveloper.com/application-development/powerruby-brings-enterprise-ruby-rails-support-ibm-i>) on 9 October 2013. Retrieved 9 May 2014.
53. Js_Escape_Map. "ActionView::Helpers::JavaScriptHelper" (<http://api.rubyonrails.org/classes/ActionView/Helpers/JavaScriptHelper.html>). *Api.rubyonrails.org*. Archived (<https://web.archive.org/web/20110722151736/http://api.rubyonrails.org/classes/ActionView/Helpers/JavaScriptHelper.html>) from the original on 22 July 2011. Retrieved 24 May 2017.
54. David (29 August 2010). "Rails 3.0: It's ready!" (<http://weblog.rubyonrails.org/2010/8/29/rails-3-0-it-s-done>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20100831063441/http://weblog.rubyonrails.org/2010/8/29/rails-3-0-it-s-done>) from the original on 31 August 2010. Retrieved 30 August 2010. "Rails 3.0 has been designed to work with Ruby 1.8.7, Ruby 1.9.2, and JRuby 1.5.2+."

55. Spastorino (1 September 2012). "Rails 3.2.0.rc2 has been released!" (<http://weblog.rubyonrails.org/2012/1/4/rails-3-2-0-rc2-has-been-released/>). *weblog.rubyonrails.org*. Archived (<https://web.archive.org/web/20120901210645/http://weblog.rubyonrails.org/2012/1/4/rails-3-2-0-rc2-has-been-released/>) from the original on 1 September 2012. Retrieved 3 September 2012.
56. "Ruby on Rails: Download" (<https://web.archive.org/web/20091212173926/http://rubyonrails.org/download>). *rubyonrails.org*. Archived from the original (<http://rubyonrails.org/download>) on 12 December 2009.
57. "Getting Started with Rails" (http://guides.rubyonrails.org/getting_started.html). *guides.rubyonrails.org*. Archived (https://web.archive.org/web/20160429222517/http://guides.rubyonrails.org/getting_started.html) from the original on 29 April 2016. Retrieved 10 March 2014.
58. David Heinemeier Hansson (7 Dec 2021) The time is right for Hotwire (<https://world.hey.com/dhh/the-time-is-right-for-hotwire-ecdb9b33>) Archived (<https://web.archive.org/web/20211220055233/https://world.hey.com/dhh/the-time-is-right-for-hotwire-ecdb9b33>) 20 December 2021 at the [Wayback Machine](#)
59. David Heinemeier Hansson (6 September 2021) Rails 7 will have three great answers to JavaScript in 2021+ (<https://world.hey.com/dhh/rails-7-will-have-three-great-answers-to-javascript-in-2021-8d68191b>) Archived (<https://web.archive.org/web/20211220055233/https://world.hey.com/dhh/rails-7-will-have-three-great-answers-to-javascript-in-2021-8d68191b>) 20 December 2021 at the [Wayback Machine](#)
60. David Heinemeier Hansson (19 April 2018) Conceptual compression means beginners don't need to know SQL — hallelujah! (<https://m.signalvnoise.com/conceptual-compression-means-beginners-dont-need-to-know-sql-hallelujah/>) Archived (<https://web.archive.org/web/20211220055233/https://m.signalvnoise.com/conceptual-compression-means-beginners-dont-need-to-know-sql-hallelujah/>) 20 December 2021 at the [Wayback Machine](#)
61. David Heinemeier Hansson (15 May 2018) RailsConf 2018: Opening Keynote: FIXME by David Heinemeier Hansson (<https://www.youtube.com/watch?v=zKyv-IGvgGE>) Archived (<https://web.archive.org/web/20211220055232/https://www.youtube.com/watch?v=zKyv-IGvgGE>) 20 December 2021 at the [Wayback Machine](#) Conceptual compression
62. David Heinemeier Hansson (31 Aug 2020) Serious security (<https://www.hey.com/security/>) Archived (<https://web.archive.org/web/20211226214333/https://www.hey.com/security/>) 26 December 2021 at the [Wayback Machine](#)
63. David Heinemeier Hansson, creator of Ruby on Rails and hey.com (19 Dec 2021) Ruby on Rails Creator Touts 7.0 as One-Person Framework, 'The Way It Used To Be' (<https://developers.slashdot.org/story/21/12/19/2043226/ruby-on-rails-creator-touts-70-as-one-person-framework-the-way-it-used-to-be>) Archived (<https://web.archive.org/web/20211220055233/https://developers.slashdot.org/story/21/12/19/2043226/ruby-on-rails-creator-touts-70-as-one-person-framework-the-way-it-used-to-be>) 20 December 2021 at the [Wayback Machine](#)
64. "'Ruby on Rails' Trademark Status" (<http://tarr.uspto.gov/servlet/tarr?regser=serial&entry=77119206>). USPTO. Archived (<https://web.archive.org/web/20071014232431/http://tarr.uspto.gov/servlet/tarr?regser=serial&entry=77119206>) from the original on 14 October 2007. Retrieved 15 December 2021.
65. "'Rails' Trademark Status" (<http://tarr.uspto.gov/servlet/tarr?regser=serial&entry=77119210>). USPTO. Archived (<https://web.archive.org/web/20071015013918/http://tarr.uspto.gov/servlet/tarr?regser=serial&entry=77119210>) from the original on 15 October 2007. Retrieved 15 December 2021.
66. "Rails Logo Trademark Status" (<http://tarr.uspto.gov/servlet/tarr?regser=serial&entry=77119208>). USPTO. Archived (<https://web.archive.org/web/20071015000411/http://tarr.uspto.gov/servlet/tarr?regser=serial&entry=77119208>) from the original on 15 October 2007. Retrieved 15 December 2021.

67. Forde, Pete (23 July 2007). "Beginning Rails: From Novice to Professional" (<https://web.archive.org/web/20070805232643/http://rethink.unspace.ca/2007/7/23/beginning-rails-from-novice-to-professional>). Archived from the original (<http://rethink.unspace.ca/2007/7/23/beginning-rails-from-novice-to-professional>) on 5 August 2007. Retrieved 1 August 2007.
68. Cooper, Peter (24 July 2007). "David Heinemeier Hansson says No to Use of Rails Logo" (<http://www.rubyinside.com/david-heinemeier-hansson-says-no-to-use-of-rails-logo-567.html>). Archived (<https://web.archive.org/web/20170825060424/http://www.rubyinside.com/david-heinemeier-hansson-says-no-to-use-of-rails-logo-567.html>) from the original on 25 August 2017. Retrieved 1 August 2007.
69. "5 Question Interview with Twitter Developer Alex Payne" (<https://web.archive.org/web/20090423002030/http://www.radicalbehavior.com/5-question-interview-with-twitter-developer-alex-payne/>). radicalbehavior.com. 29 March 2007. Archived from the original (<http://www.radicalbehavior.com/5-question-interview-with-twitter-developer-alex-payne/>) on 23 April 2009. Retrieved 4 November 2014. *"By various metrics Twitter is the biggest Rails site on the net right now. Running on Rails has forced us to deal with scaling issues - issues that any growing site eventually contends with – far sooner than I think we would on another framework."*
70. Steve Jenson; Alex Payne & Robey Pointer interview (3 April 2009). "Twitter on Scala" (http://www.artima.com/scalazine/articles/twitter_on_scala.html). artima.com. Archived (https://web.archive.org/web/20090619164149/http://www.artima.com/scalazine/articles/twitter_on_scala.html) from the original on 19 June 2009. Retrieved 18 July 2009. *"We had a Ruby-based queuing system that we used for communicating between the Rails front ends and the daemons, and we ended up replacing that with one written in Scala. The Ruby one actually worked pretty decently in a normal steady state, but the startup time and the crash behavior were undesirable."*
71. "Twitter jilts Ruby for Scala" (https://www.theregister.co.uk/2009/04/01/twitter_on_scala/). theregister.co.uk. 1 April 2009. Archived (https://web.archive.org/web/20170812232515/http://www.theregister.co.uk/2009/04/01/twitter_on_scala/) from the original on 12 August 2017. Retrieved 18 July 2009. *"By the end of this year, Payne said, Twitter hopes to have its entire middleware infrastructure and its APIs ported to the new language. Ruby will remain, but only on the front end. "We're still happy with Rails for building user facing features... performance-wise, it's fine for people clicking around web pages. It's the heavy lifting, asynchronous processing type of stuff that we've moved away from."*
72. Ryan King (25 September 2009). "Twitter on Ruby" (<http://blog.evanweaver.com/articles/2009/09/24/ree/>). Evan Weaver. Archived (<https://web.archive.org/web/20090927063142/http://blog.evanweaver.com/articles/2009/09/24/ree/>) from the original on 27 September 2009. Retrieved 29 September 2009. *"We use Scala for a few things at Twitter, but the majority of the site is Ruby."*
73. "Twitter Search is Now 3x Faster" (<https://blog.twitter.com/2011/twitter-search-now-3x-faster>). Twitter. Retrieved 6 June 2014.
74. "I won't let you pay me for my open source" (<https://world.hey.com/dhh/i-won-t-let-you-pay-me-for-my-open-source-d7cf4568>). Archived (<https://web.archive.org/web/20211215075328/https://world.hey.com/dhh/i-won-t-let-you-pay-me-for-my-open-source-d7cf4568>) from the original on 15 December 2021. Retrieved 15 December 2021.
75. "Why we're sticking with Ruby on Rails" (<https://about.gitlab.com/blog/2022/07/06/why-were-sticking-with-ruby-on-rails/>). GitLab. Archived (<https://web.archive.org/web/20220829003627/https://about.gitlab.com/blog/2022/07/06/why-were-sticking-with-ruby-on-rails/>) from the original on 29 August 2022. Retrieved 29 August 2022.
76. "Here's Why Ruby On Rails Is Hot" (https://web.archive.org/web/20111221011816/http://articles.businessinsider.com/2011-05-11/tech/30035869_1_ruby-rails-custom-software). Business Insider. Archived from the original (http://articles.businessinsider.com/2011-05-11/tech/30035869_1_ruby-rails-custom-software) on 21 December 2011. Retrieved 10 February 2012.

77. "Ruby on Rails Usage Statistics" (<http://trends.builtwith.com/framework/Ruby-on-Rails>). *trends.builtwith.com*. Archived (<https://web.archive.org/web/20120212155607/http://trends.builtwith.com/framework/Ruby-on-Rails>) from the original on 12 February 2012. Retrieved 4 January 2016.
78. "Ruby on Rails Token Usage Statistics" (<http://trends.builtwith.com/framework/Ruby-on-Rails-Token>). *trends.builtwith.com*. Archived (<https://web.archive.org/web/20160314105340/http://trends.builtwith.com/framework/Ruby-on-Rails-Token>) from the original on 14 March 2016. Retrieved 4 January 2016.
79. Protalinski, Emil. "How GitHub handled getting hacked" (<https://www.zdnet.com/article/how-github-handled-getting-hacked/>). *Zdnet.com*. Retrieved 24 May 2017.
80. Brook, Chris (25 September 2013). "Security Issue in Ruby on Rails Could Expose Cookies" (<http://threatpost.com/security-issue-in-ruby-on-rails-could-expose-cookies/102413>). *Threat Post*. The Kaspersky Lab Security News Service. Retrieved 4 June 2014.
81. "Patching up Web applications" (<https://news.mit.edu/2016/patching-web-applications-0415>). *MIT News*. 15 April 2016. Archived (<https://web.archive.org/web/20190727223529/http://news.mit.edu/2016/patching-web-applications-0415>) from the original on 27 July 2019. Retrieved 21 April 2016.

Bibliography

- Ruby, Sam; Thomas, Dave; Hansson, David (28 March 2009). *Agile Web Development with Rails* (<https://archive.org/details/agilewebdevelopm0000unse>) (Third ed.). Pragmatic Bookshelf. p. 850. ISBN 978-1-934356-16-6.
- Laurent, Simon St.; Dumbill, Edd (28 November 2008). *Learning Rails* (<https://archive.org/details/learningrails00simo>) (First ed.). O'Reilly Media. p. 442. ISBN 978-0-596-51877-6.
- Lenz, Patrick (1 May 2008). *Simply Rails 2* (<https://archive.org/details/simplyrails2000patr>) (Second ed.). SitePoint. p. 450 (<https://archive.org/details/simplyrails2000patr/page/450>). ISBN 978-0-9804552-0-5.
- Tate, Bruce; Hibbs, Curt (22 August 2006). *Ruby on Rails: Up and Running* (<http://oreilly.com/catalog/9780596101329/>) (First ed.). O'Reilly Media. p. 182. ISBN 0-596-10132-5. Archived (<https://web.archive.org/web/20090627153614/http://oreilly.com/catalog/9780596101329/>) from the original on 27 June 2009. Retrieved 12 July 2009.
- Holzner, Steve (29 November 2006). *Beginning Ruby on Rails* (<https://web.archive.org/web/20120423022010/http://www.wrox.com/WileyCDA/WroxTitle/Beginning-Ruby-on-Rails.productCd-0470069155.html>) (First ed.). Wrox. p. 380. ISBN 0-470-06915-5. Archived from the original (<http://www.wrox.com/WileyCDA/WroxTitle/Beginning-Ruby-on-Rails.productCd-0470069155.html>) on 23 April 2012. Retrieved 12 July 2009.
- Allan Hardy, Jeffrey; Carneiro Jr, Cloves; Catlin, Hampton (20 July 2007). *Beginning Ruby on Rails E-Commerce: From Novice to Professional* (<https://web.archive.org/web/20100917191031/http://apress.com/book/view/9781590597361>) (First ed.). Wrox. p. 361. ISBN 978-1-59059-686-9. Archived from the original (<http://www.apress.com/book/view/9781590597361>) on 17 September 2010. Retrieved 12 July 2009.
- Clark, Mike (15 May 2008). *Advanced Rails Recipes* (<https://archive.org/details/advancedrailsrec0000clar>) (First ed.). Pragmatic Bookshelf. p. 464. ISBN 978-0-9787392-2-5.

External links

- Official website (<https://rubyonrails.org>) 
 - Guides (<http://guides.rubyonrails.org/>)

- [API \(http://api.rubyonrails.org/\)](http://api.rubyonrails.org/)
 - [rails \(https://github.com/rails/rails\)](https://github.com/rails/rails) on [GitHub](#)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Ruby_on_Rails&oldid=1352671642"